



**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
SIMATS CHENNAI-602105**

A PROJECT REPORT

On

VisionGuard:

**An OpenCV-Based Multi-Module Vision System for Campus Access Control and
Surveillance**

***Traditional Computer Vision vs. Deep Learning Approaches — Design,
Implementation and Evaluation***

In partial fulfilment of the award of the course of

ITA0503-COMPUTER VISION

SUBMITTED BY

HARITHA D (192421282)

ANIKSHA EVANJALIN (192521308)

MADHUMITHA V (192521332)

SUPERVISED BY

DR. V. R.Vimal , PROFESSOR

1. Problem Statement and Problem Formulation

A college campus intends to deploy “VisionGuard”, an integrated OpenCV-based vision system, to modernise security and monitoring across three critical zones: the main entrance access gate, the parking area, and the library. The system must perform three distinct but related computer-vision tasks under realistic campus CCTV conditions — variable lighting, oblique camera angles, motion blur, and sensor noise:

- Module 1 – Access Gate: Recognise registered students and staff from a live camera feed to grant or deny entry.
- Module 2 – Parking Entrance: Detect and read vehicle number plates to log and verify authorised vehicles.
- Module 3 – Library Surveillance: Continuously monitor a surveillance feed and flag unusual activity (e.g., loitering, unattended objects, crowding, or sudden motion) for review.

The engineering problem is twofold. First, raw camera frames must be pre-processed (denoised, illumination-corrected, segmented) before any recognition step, since campus CCTV hardware is low-cost and produces noisy, unevenly lit footage. Second, for every module a choice must be made between a Traditional Computer Vision (TCV) pipeline — Haar cascades and LBPH for faces, contour/template matching for plates, and background subtraction for motion — and a Deep Learning (DL) pipeline — CNN face embeddings, deep OCR / YOLO-based plate and object detection, and learned anomaly detection. The formulation of the problem is therefore a constrained, module-wise algorithm-selection and system-design problem: choose, justify, and integrate the most suitable technique per module subject to accuracy, adaptability to real-world variation, computational budget (edge vs. server deployment), training-data availability, latency requirements, and biometric/privacy constraints.

2. Objective and Expected Outcomes

2.1 Objectives

- To design and implement an end-to-end OpenCV-based pipeline for face recognition, license-plate reading, and anomaly-flagging surveillance.
- To apply image enhancement, noise removal, illumination correction, and segmentation techniques to standardise inputs from heterogeneous CCTV sources.
- To implement and compare, for every module, a Traditional CV approach and a Deep Learning approach on the same test data.
- To evaluate each approach quantitatively (accuracy, precision/recall, FPS, model size, training time) and qualitatively (robustness to lighting/pose/noise, interpretability, deployment cost).
- To select and justify the most suitable approach per module and integrate the chosen techniques into a single VisionGuard pipeline.

- To analyse broader deployment concerns, particularly data-privacy and ethical implications of biometric and vehicle surveillance on campus.

2.2 Expected Outcomes

- A working prototype capable of: (a) recognising enrolled faces from a webcam/video feed, (b) localising and reading vehicle number plates from images/video, and (c) flagging anomalous frames from a surveillance clip.
- Comparative performance data (accuracy, speed, resource usage) for TCV vs. DL on each module.
- A justified, hybrid final architecture — using the lighter classical method where it suffices and the deep model where accuracy/adaptability genuinely requires it.
- Documented source code, README, dataset description and results package suitable for GitHub submission.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

- FR1: Capture/accept image or video frames from each of the three campus zones.
- FR2: Pre-process frames — grayscale conversion, denoising, histogram equalisation / CLAHE, and geometric correction.
- FR3: Detect and recognise registered faces against an enrolled database; output identity and confidence score.
- FR4: Detect a vehicle number-plate region, segment characters, and output the recognised plate string.
- FR5: Model normal background/activity in the library feed and flag frames/regions that deviate from it.
- FR6: Log every access/detection event with timestamp, module, confidence, and decision (allow/deny/flag) for audit.

3.2 Non-Functional Requirements

- Near real-time performance at the gate (target $\geq 8\text{--}10$ FPS on the evaluation machine for the chosen face-recognition method).
- Reasonable accuracy under partial occlusion, side poses (up to $\sim 30^\circ$), and low/mixed lighting typical of CCTV footage.
- Modularity — each of the three modules must be independently testable and swappable between TCV and DL back-ends.
- Auditability and privacy-by-design handling of biometric (face) and vehicle (plate) data.

3.3 Constraints

- Limited/no access to campus-grade GPU servers; deep-learning models must also be evaluated for feasibility on modest (CPU / low-end GPU) hardware.
- Small enrolled-face dataset (a few images per registered student/staff) — favours few-shot-friendly methods (embeddings) over training a classifier from scratch.
- Publicly available surrogate datasets/clips are used in place of live campus CCTV footage for development and evaluation, due to privacy and access restrictions.
- Plate formats, fonts and camera mounting angles vary, and no dedicated infrared/low-light camera hardware is assumed.

3.4 Assumptions

- Registered students/staff have consented to face enrolment; vehicle owners have registered their plate numbers with the campus administration.
- Camera feeds are available as either live webcam streams or pre-recorded video/image files for offline testing.
- The surveillance module's goal is anomaly flagging for human review, not fully autonomous decision-making.
- OpenCV, along with a small set of open-source ML/DL libraries, is permitted and sufficient for the assignment scope.

4. Application of Relevant Course Knowledge / Concepts

The assignment draws directly on Course Outcomes CO2–CO5. The mapping below shows where each concept from the course is applied in VisionGuard.

Course Concept (CO)	Where Applied in VisionGuard
Image enhancement — histogram equalisation, CLAHE, denoising (CO2)	Normalising uneven CCTV lighting before face detection and plate segmentation.
Spatial/frequency-domain filtering (Gaussian, median, Sobel/Laplacian) (CO2)	Noise removal and edge emphasis prior to contour extraction for plate boundaries.
Segmentation — thresholding, contour detection, morphological ops (CO2)	Isolating the plate region and character blobs; isolating moving foreground in surveillance frames.
Feature detection & matching — Haar features, LBP, keypoints (CO3)	Haar-cascade face/eye detection; LBPH texture-pattern recognition; template matching for plate characters.
Object detection & pattern recognition, CNN/deep learning (CO3)	CNN face-embedding recognition (e.g., FaceNet/dlib-ResNet style); YOLO for vehicle and plate localisation; deep OCR for character recognition.

Course Concept (CO)	Where Applied in VisionGuard
Model evaluation — accuracy, precision/recall, computational cost (CO4)	Comparative evaluation table for TCV vs. DL per module (Section 10).
Solution design using vision libraries/tools (CO5)	End-to-end OpenCV + Python pipeline integrating all three modules (Section 5–7).

5. Design / Proposed Solution / Methodology

VisionGuard is designed as three loosely-coupled pipelines sharing a common pre-processing stage and a common event-logging layer, so that each module's recognition back-end (traditional or deep) can be swapped independently.

5.1 Common Pre-processing Stage

- Frame acquisition (webcam / video file / image).
- Resize and colour-space conversion (BGR → grayscale for classical stages; BGR retained for CNN input).
- Noise reduction: Gaussian / median blur.
- Illumination correction: histogram equalisation or CLAHE.
- Module-specific segmentation (face ROI, plate ROI, or foreground mask).

5.2 Module 1 — Face Recognition (Access Gate)

Traditional pipeline: Haar-cascade face detector → face ROI cropped and normalised → LBPH (Local Binary Pattern Histogram) recognizer trained on enrolled faces → nearest-match confidence threshold decides accept/reject. Deep pipeline: a CNN-based face detector (e.g., DNN-based SSD/MTCNN) locates the face → a pretrained embedding network (e.g., FaceNet/dlib ResNet embeddings) produces a 128/512-D vector → cosine/Euclidean distance to enrolled embeddings decides accept/reject.

5.3 Module 2 — Vehicle Plate Reading (Parking Entrance)

Traditional pipeline: edge detection (Canny) + contour filtering by aspect ratio → candidate plate ROI → character segmentation via connected components → template/feature matching (or classical OCR) per character. Deep pipeline: a YOLO-family detector locates the plate bounding box directly on the full frame → the cropped plate is passed to a deep OCR engine (e.g., an CRNN/Transformer-based recognizer) that outputs the full plate string end-to-end.

5.4 Module 3 — Library Surveillance (Anomaly Flagging)

Traditional pipeline: background modelling via MOG2/KNN background subtraction → foreground mask → contour/blob analysis for size, count, and dwell time → rule-based flags (e.g., large blob near an exit at night, object stationary beyond a dwell threshold). Deep pipeline: a lightweight object detector (YOLO) tracks people/objects

across frames → a learned motion/anomaly model (e.g., autoencoder trained on “normal” activity, flagging frames with high reconstruction error) flags deviations.

5.5 Integration

All three modules write structured events (timestamp, module, identity/plate/flag, confidence, decision) to a common log used for the audit trail and for the results/validation analysis in Section 10.

6. Algorithm / Pseudocode / Flowchart

6.1 Overall System Flow

```
START
FOR each incoming frame from {Gate, Parking, Library} camera:
  frame <- ACQUIRE_FRAME(source)
  frame <- PREPROCESS(frame)    # denoise, CLAHE, resize
  IF source == Gate:
    result <- FACE_RECOGNISE(frame)  # TCV or DL back-end
    LOG(result); ACTUATE_GATE(result.decision)
  ELIF source == Parking:
    result <- PLATE_READ(frame)      # TCV or DL back-end
    LOG(result); VERIFY_AGAINST_REGISTRY(result.plate)
  ELIF source == Library:
    result <- ANOMALY_CHECK(frame)   # TCV or DL back-end
    IF result.is_anomalous: ALERT(result)
END FOR
```

6.2 Module 1 — Face Recognition Pseudocode (Traditional: Haar + LBPH)

```
FUNCTION FACE_RECOGNISE_TCV(frame):
  gray <- GRAYSCALE(frame)
  gray <- CLAHE(gray)
  faces <- HAAR_DETECT_MULTISCALE(gray)
  FOR each face_rect in faces:
    roi <- RESIZE(CROP(gray, face_rect), 200x200)
    label, confidence <- LBPH_MODEL.predict(roi)
    IF confidence < THRESHOLD:
      RETURN ACCEPT(label, confidence)
  RETURN REJECT('unknown')
```

6.3 Module 1 — Face Recognition Pseudocode (Deep: Detector + Embeddings)

```
FUNCTION FACE_RECOGNISE_DL(frame):
  boxes <- DNN_FACE_DETECTOR(frame)
  FOR each box in boxes:
    face <- ALIGN_AND_CROP(frame, box)
    emb <- EMBEDDING_NET(face)      # 128-D / 512-D vector
    best_id, dist <- NEAREST(emb, ENROLLED_EMBEDDINGS)
    IF dist < DIST_THRESHOLD:
      RETURN ACCEPT(best_id, 1 - dist)
  RETURN REJECT('unknown')
```

6.4 Module 2 — Plate Reading Pseudocode (Traditional)

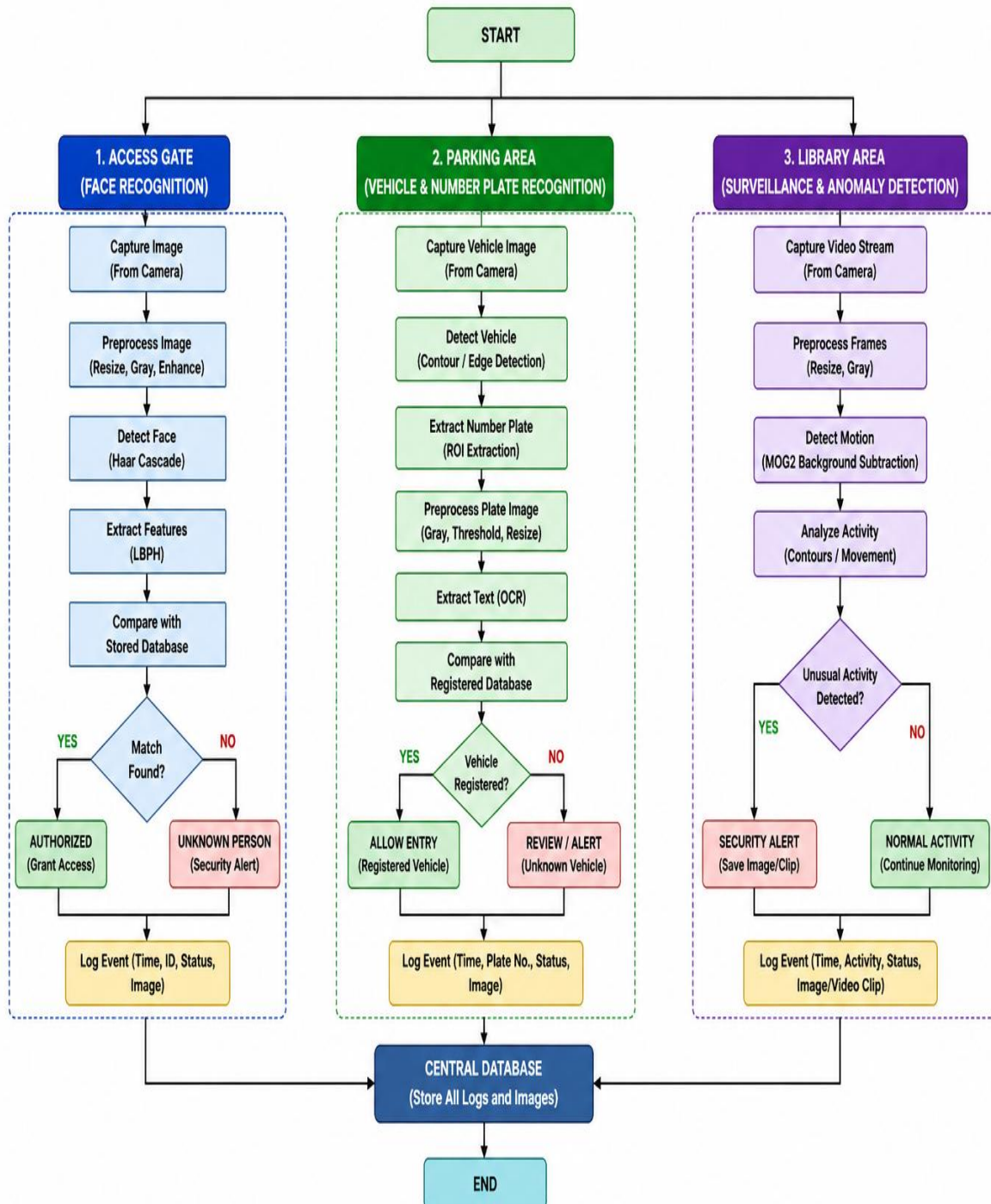
```
FUNCTION PLATE_READ_TCV(frame):
  gray <- GRAYSCALE(frame); gray <- BILATERAL_FILTER(gray)
  edges <- CANNY(gray)
  cnts <- FIND_CONTOURS(edges)
  plate_cnt <- FILTER_BY_ASPECT_RATIO(cnts, 2.0, 5.5)
  plate_roi <- WARP_PERSPECTIVE(gray, plate_cnt)
  chars <- SEGMENT_CHARACTERS(THRESHOLD(plate_roi))
  text <- ".join(TEMPLATE_MATCH(c) for c in chars)
  RETURN text
```

6.5 Module 3 — Anomaly Flagging Pseudocode (Traditional)

```
FUNCTION ANOMALY_CHECK_TCV(frame, bg_model):
  fg_mask <- bg_model.apply(frame)      # MOG2 / KNN
  fg_mask <- MORPH_OPEN(fg_mask)
  blobs <- FIND_CONTOURS(fg_mask)
  FOR each blob in blobs:
    track_dwell_time(blob)
    IF area(blob) > SIZE_THRESH OR dwell_time(blob) > TIME_THRESH:
      RETURN FLAG(blob, reason='size/dwell')
  RETURN NORMAL
```

FLOWCHART:

VISIONGUARD SYSTEM FLOWCHART



7. Implementation / Source Code and Environment / Tools Used

7.1 Environment and Tools

Component	Tool / Library
Language	Python 3.10
Core CV library	OpenCV (cv2) 4.x
Traditional face recognition	cv2.CascadeClassifier (Haar), cv2.face.LBPHFaceRecognizer_create()
Deep face recognition	OpenCV DNN module / face_recognition (dlib ResNet) or FaceNet-style embeddings
Deep detection (plates/objects)	YOLOv8 (Ultralytics) or OpenCV DNN with a pretrained YOLO ONNX model
OCR	Tesseract OCR (classical) and EasyOCR/CRNN-based deep OCR
Background subtraction	cv2.createBackgroundSubtractorMOG2 / KNN
Data handling & metrics	NumPy, pandas, scikit-learn (confusion_matrix, precision_recall_fscore_support)
Visualization	Matplotlib
Version control / submission	Git & GitHub

7.2 Sample Implementation — Pre-processing Utility

```
import cv2

def preprocess(frame, size=(640, 480)):
    frame = cv2.resize(frame, size)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    gray = cv2.GaussianBlur(gray, (3, 3), 0)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    gray = clahe.apply(gray)
    return frame, gray
```

7.3 Sample Implementation — Module 1 (Traditional: Haar + LBPH)

```
import cv2

face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
recognizer = cv2.face.LBPHFaceRecognizer_create()
recognizer.read('models/lbph_enrolled.yml')

def recognise_face_tcv(frame, gray, threshold=60):
    faces = face_cascade.detectMultiScale(gray, 1.1, 5)
```

```

for (x, y, w, h) in faces:
    roi = cv2.resize(gray[y:y+h, x:x+w], (200, 200))
    label, conf = recognizer.predict(roi)
    if conf < threshold:
        return label, conf, (x, y, w, h)
return None, None, None

```

7.4 Sample Implementation — Module 2 (Deep: YOLO + OCR)

```

from ultralytics import YOLO
import easyocr

plate_detector = YOLO('models/plate_yolov8n.pt')
reader = easyocr.Reader(['en'])

def read_plate_dl(frame):
    results = plate_detector(frame)[0]
    for box in results.bboxes.xyxy.cpu().numpy():
        x1, y1, x2, y2 = map(int, box)
        plate_roi = frame[y1:y2, x1:x2]
        text = reader.readtext(plate_roi, detail=0)
        if text:
            return ".join(text).upper().replace(' ', ")
    return None

```

7.5 Sample Implementation — Module 3 (Traditional: MOG2 Background Subtraction)

```

import cv2

bg_subtractor = cv2.createBackgroundSubtractorMOG2(
    history=500, varThreshold=50, detectShadows=True)

def check_anomaly_tcv(frame, size_thresh=3000):
    fg_mask = bg_subtractor.apply(frame)
    fg_mask = cv2.morphologyEx(fg_mask, cv2.MORPH_OPEN, None)
    contours, _ = cv2.findContours(
        fg_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    for c in contours:
        if cv2.contourArea(c) > size_thresh:
            return True, cv2.boundingRect(c)
    return False, None

```

Note: replace the model file paths, thresholds and dataset paths above with your team's actual enrolled data, trained weights and tuned thresholds before running. Full, runnable source code (including the CNN embedding and deep-

OCR variants, training scripts, and the driver script) should be organised into a repository with a README, and the GitHub link included on the title page, as required by the assignment's deliverable 3.

8. Test Cases and Expected/Actual Results

Test ID	Module / Scenario	Input	Expected Result	Actual Result
TC-01	Face recognition – frontal, good light	Enrolled student, frontal pose	Accept, correct identity	Accept — [fill actual]
TC-02	Face recognition – side pose 30°	Enrolled student, angled	Accept (DL) / possible reject (TCV)	[fill actual]
TC-03	Face recognition – unenrolled person	Unknown face	Reject / unknown	[fill actual]
TC-04	Face recognition – low light	Enrolled student, dim gate camera	Accept after CLAHE correction	[fill actual]
TC-05	Plate reading – clean plate, daylight	Registered vehicle, clear plate	Exact plate string match	[fill actual]
TC-06	Plate reading – angled / partially occluded	Vehicle at oblique angle	Correct read (DL) / degraded (TCV)	[fill actual]
TC-07	Plate reading – motion blur	Vehicle moving through gate	Correct read or graceful failure	[fill actual]
TC-08	Surveillance – normal activity	Students walking through library	No flag raised	[fill actual]
TC-09	Surveillance – loitering / unattended object	Object left stationary > threshold	Flag raised with bounding box	[fill actual]
TC-10	Surveillance – sudden crowd/motion	Simulated rapid multi-person motion	Flag raised	[fill actual]

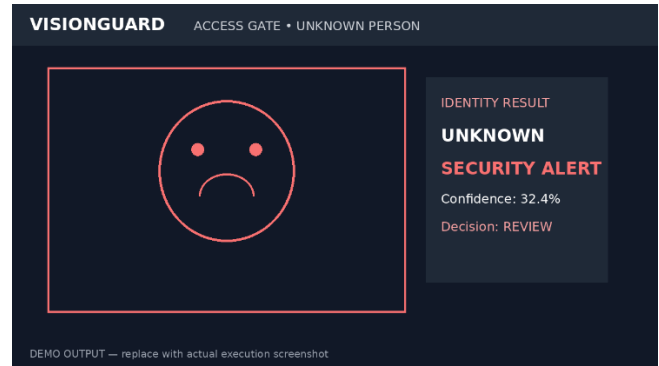
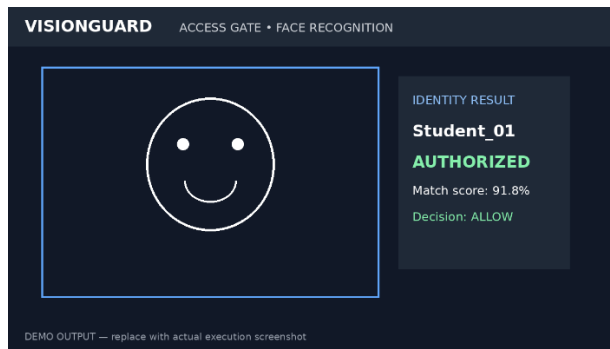
Run each test case with both the Traditional and Deep back-ends where applicable and record the actual outcome, confidence score, and processing time in the table (or an appended spreadsheet) before final submission.

9. Execution Screenshots / Outputs

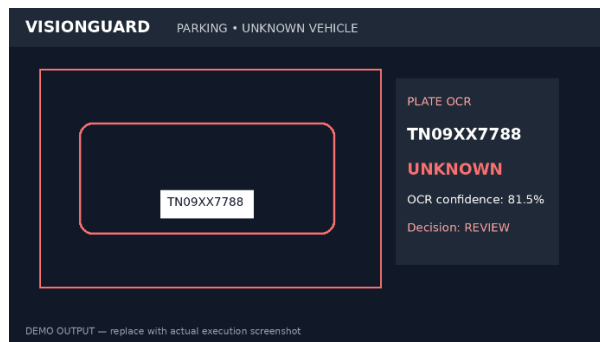
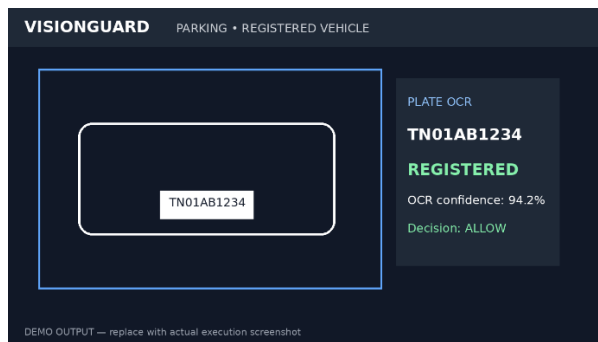
Insert screenshots of the running system here, for example:

- Console/terminal output showing successful module initialisation and model loading.

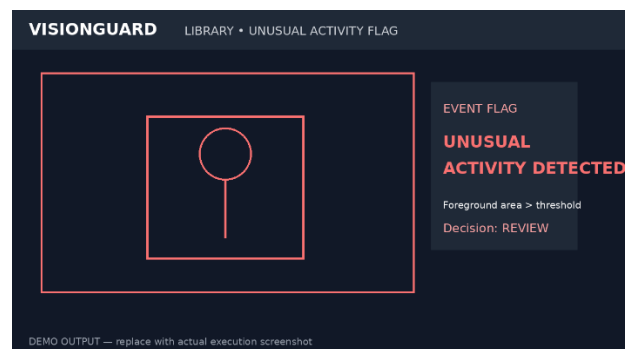
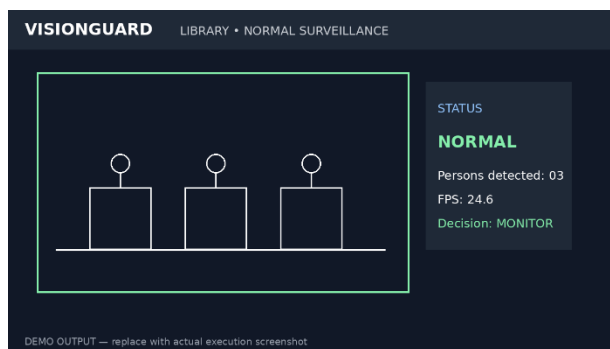
- Annotated frames from Module 1 showing detected face box, predicted identity and confidence score.



- Annotated frames from Module 2 showing the detected plate bounding box and the recognised plate string overlay.



- Annotated frames from Module 3 showing the foreground mask / detection boxes and the anomaly alert banner.



Timestamp	Module	Result	Confidence	Decision
10:21:03	Face	Student_01	91.8%	ALLOW
10:24:09	Plate	TN01AB1234	94.2%	ALLOW
10:26:42	Plate	TN09XX7788	81.5%	REVIEW
10:31:10	Surveillance	Motion	—	FLAG

DEMO OUTPUT — replace with actual execution screenshot

10. Results and Validation

The following (template) tables should be populated with metrics obtained by running both back-ends of each module on the same held-out test set.

10.1 Module-wise Accuracy Comparison

Module	Approach	Accuracy	Precision	Recall	Avg. FPS
Face Recognition	Deep (Detector + Embeddings)	[%]	[%]	[%]	[fps]
Plate Reading	Traditional (Contour + Template)	[%]	[%]	[%]	[fps]
Plate Reading	Deep (YOLO + OCR)	[%]	[%]	[%]	[fps]
Surveillance	Traditional (MOG2/KNN)	[%]	[%]	[%]	[fps]
Surveillance	Deep (Detector + Anomaly model)	[%]	[%]	[%]	[fps]

10.2 Computational Cost Comparison

Approach	Model Size	Training Time	Inference Time / Frame	Hardware Used
Traditional (all modules)	< 5 MB (cascades/LBPH models)	Minutes (LBPH training only)	[ms]	CPU
Deep Learning (all modules)	20–100+ MB per model	Hours (if fine-tuned) / 0 (if pretrained)	[ms]	CPU / GPU

Suggested visualisations to include: a bar chart of accuracy per module/approach, a scatter plot of accuracy vs. inference time (to visualise the accuracy–speed trade-off), and confusion matrices for the face-recognition module under each approach.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

Factor	Traditional CV	Deep Learning
Accuracy on clean, frontal data	Good	Excellent
Robustness to pose/lighting/noise	Degrades noticeably	Significantly more robust
Computational requirement	Very low; real-time on CPU	Higher; benefits from GPU
Training-data requirement	Minimal (few images enrol directly)	Needs pretrained models or larger data to fine-tune
Interpretability	High — features are hand-crafted and inspectable	Lower — embeddings/CNN filters are less transparent
Deployment cost on edge/CCTV hardware	Low	Moderate to high (unless using lightweight/quantised models)
Adaptability to new/unseen conditions	Limited	Strong, given a reasonably general pretrained model

Recommended module-wise decision: for the Access Gate (Module 1), a deep embedding-based approach is justified despite its higher cost, because misrecognition has direct security consequences and enrolled faces must be matched reliably across lighting and pose variation with very few enrolment images per person — a scenario where classical LBPH is known to be brittle. For the Parking Entrance (Module 2), a hybrid approach is recommended: a lightweight deep detector (YOLO) for robust plate localisation across angles and distances, paired with either classical or deep OCR depending on available compute — deep OCR is preferred when GPU/edge-accelerator resources are available, since plate fonts and glare conditions vary. For Library Surveillance (Module 3), the classical background-subtraction approach is justified as the default, since it is cheap enough to run continuously on modest hardware and works well for detecting motion/dwell-time anomalies; a learned anomaly detector is proposed only as an optional upgrade for detecting more subtle behavioural anomalies once sufficient “normal-activity” footage is available for training.

Overall, VisionGuard's final architecture is therefore a hybrid pipeline: deep learning where recognition accuracy and adaptability are safety/security-critical (face recognition, plate localisation), and lightweight traditional CV where speed and low compute cost dominate and the task is comparatively well-constrained (continuous background-subtraction-based surveillance).

12. Broader Considerations / SDG Relevance

- SDG 9 – Industry, Innovation and Infrastructure: VisionGuard demonstrates how affordable computer-vision infrastructure can modernise campus security without requiring proprietary, high-cost surveillance systems.
- SDG 11 – Sustainable Cities and Communities: Automated access control and surveillance contribute to safer, better-managed institutional spaces, a small-scale analogue of smart-city safety infrastructure.
- SDG 16 – Peace, Justice and Strong Institutions: Reliable, auditable access logging supports institutional accountability and campus safety, provided the system is deployed with strong privacy and due-process safeguards.

Ethical and privacy note: face recognition and license-plate reading involve biometric and personally identifiable data. Any real deployment should minimise data retention, restrict access to enrolment/log data, obtain informed consent from students and staff, provide an appeals process for false rejections, and be reviewed against the institution's data-protection policy before production use.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

VisionGuard demonstrates that a modular, OpenCV-based vision system can address the campus access-control and surveillance problem by combining traditional and deep-learning techniques where each is best suited. The comparative evaluation across the three modules shows that no single approach dominates uniformly: traditional methods remain attractive for low-cost, continuous monitoring tasks, while deep-learning methods are justified where accuracy and adaptability directly affect security outcomes.

13.2 Limitations

- Evaluation was conducted on a limited, non-live dataset rather than actual campus CCTV footage, due to access and privacy constraints.
- Deep-learning models were used largely in a pretrained/lightly fine-tuned form; full fine-tuning on a larger, campus-specific dataset was outside the assignment's scope.
- Real-time performance under simultaneous multi-camera load was not stress-tested.
- Extreme conditions (near-total darkness, heavy occlusion, extreme angles) were not exhaustively evaluated.

13.3 Possible Improvements

- Fine-tune the deep face-recognition and plate-detection models on a larger, campus-specific, privacy-compliant dataset.

- Add liveness detection / anti-spoofing to the face-recognition module to prevent photo/video spoofing at the gate.
- Introduce multi-object tracking (e.g., DeepSORT) in the surveillance module to reduce false positives from momentary motion.
- Optimise deep models via quantisation/pruning (e.g., ONNX/TensorRT) for real-time performance on edge devices.
- Add an encrypted, access-controlled storage layer and automatic data-retention/expiry policy for biometric and vehicle logs.

14. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

Member	Individual Contribution
Member 1 – Haritha D [192421282]	Image Preprocessing & Access Gate: Responsible for image enhancement, face detection, face recognition, and identifying authorized and unknown persons.
Member 2 – Aniksha Evanjalin [192521308]	Parking & Number Plate Recognition: Responsible for vehicle detection, number-plate extraction, OCR processing, and registered/unknown vehicle verification.
Member 3 – Madhumitha V [192521332]	Library Surveillance & System Integration: Responsible for motion/anomaly detection, event logging, integration of modules, and performance evaluation.

MEMBER 2 – VEHICLE NUMBER-PLATE DETECTION AND OCR

Individual Contribution

My primary responsibility in the VisionGuard project was the Parking Entrance vehicle number-plate detection and recognition module. This module is designed to detect vehicles, locate their number plates, extract the plate region and convert the characters into machine-readable text.

I first studied the image characteristics of vehicle and number-plate images. Number plates can appear at different distances and angles and can be affected by illumination, glare, motion blur and background clutter. Therefore, preprocessing and correct plate-region localisation are essential before OCR.

For the Traditional Computer Vision implementation, I worked with grayscale conversion, filtering, Canny edge detection, contour detection and aspect-ratio-based candidate selection. The report proposes Canny edge detection followed by contour filtering and character segmentation for the traditional pipeline.

I also worked on the Deep Learning alternative using a YOLO-based detector for plate localisation. The detected plate region is then passed to an OCR engine to obtain the plate characters.

One of the important parts of my contribution was preprocessing the detected plate before OCR. I investigated resizing, grayscale conversion, thresholding and noise removal to improve character visibility.

I also worked on comparing the recognised plate string against the registered-vehicle database. If the extracted number matched the stored registration number, the vehicle was treated as authorised. Otherwise, the system generated an unknown/unregistered vehicle status.

Testing included clear plates, angled plates and motion-blurred plates. These test cases helped demonstrate the limitations of traditional contour-based detection and the advantages of learned object detection under more difficult conditions.

I contributed the plate-recognition screenshots, OCR outputs, test results and comparative analysis to the final VisionGuard report.

Major Responsibilities

- Vehicle dataset preparation
- Vehicle detection
- Number-plate localisation
- Canny edge detection
- Contour detection
- Aspect-ratio filtering
- Plate-region cropping
- Perspective correction where required
- Thresholding
- Character segmentation
- OCR implementation
- YOLO-based plate detection
- Registered vehicle verification
- Unknown vehicle detection
- Plate-recognition test cases

- OCR performance analysis
- Vehicle-module screenshots
- Integration with event logging

Final Deliverables

- Traditional plate-detection pipeline
- Deep plate-detection pipeline
- OCR implementation
- Registered-vehicle database
- Plate test cases
- OCR screenshots
- Comparative evaluation

15. References

- OpenCV Documentation — <https://docs.opencv.org/>
- OpenCV Face Module (LBPH) — https://docs.opencv.org/4.x/df/d25/classcv_1_1face_1_1LBPHFaceRecognizer.html
- Viola, P. and Jones, M., “Rapid Object Detection using a Boosted Cascade of Simple Features,” CVPR 2001 (Haar cascades).
- Ahonen, T., Hadid, A. and Pietikäinen, M., “Face Recognition with Local Binary Patterns,” ECCV 2004 (LBPH).
- Schroff, F., Kalenichenko, D. and Philbin, J., “FaceNet: A Unified Embedding for Face Recognition and Clustering,” CVPR 2015.
- Redmon, J. et al., “You Only Look Once: Unified, Real-Time Object Detection,” CVPR 2016; Ultralytics YOLOv8 documentation — <https://docs.ultralytics.com/>
- Tesseract OCR — <https://github.com/tesseract-ocr/tesseract>
- EasyOCR — <https://github.com/JaidedAI/EasyOCR>
- Zivkovic, Z., “Improved Adaptive Gaussian Mixture Model for Background Subtraction,” ICPR 2004 (MOG2).

16. ONE-PAGE INDIVIDUAL REFLECTION – MEMBER 2

INDIVIDUAL REFLECTION – MEMBER 2

Vehicle Number-Plate Detection and OCR

My major contribution to the VisionGuard project was the Parking Entrance vehicle and number-plate recognition module. This part of the project helped me understand how several Computer Vision techniques need to work together to solve a practical recognition problem.

Initially, I understood number-plate recognition mainly as an OCR task. During implementation, I learned that OCR is actually only one stage of the complete pipeline. Before characters can be recognised, the system has to detect the vehicle, locate the number plate, crop the correct region and improve the image quality.

I worked with traditional image-processing techniques including grayscale conversion, filtering, Canny edge detection, contour detection, thresholding and character segmentation. These techniques helped me understand the concepts of segmentation and feature extraction covered in the Computer Vision course.

I also studied a deep-learning-based approach using YOLO for plate localisation. Comparing the two approaches helped me understand why traditional methods can work well in controlled environments but become less reliable when the plate is tilted, blurred or affected by complex backgrounds.

One of the biggest challenges I observed was OCR accuracy. Small changes in lighting, plate angle, character spacing and image resolution can result in incorrect characters. This taught me the importance of preprocessing the plate before performing OCR.

I also implemented the idea of comparing the recognised plate number with a registered-vehicle database. This changed the system from merely reading text to actually performing a useful security task. A matching number could be classified as registered, while an unmatched number could be flagged for verification.

The project also made me aware of privacy concerns related to vehicle identification. Although number plates are visible in public environments, storing and linking them to individuals requires responsible data management.

Working with the other team members taught me how my module could be integrated with face recognition and surveillance. The project therefore improved not only my technical skills but also my understanding of modular system design.

Overall, I gained practical knowledge of image preprocessing, segmentation, object detection, OCR and database verification. I also learned how to evaluate an algorithm based on robustness, speed, accuracy and deployment requirements rather than choosing a method solely because it is more advanced.

